

1 Q5: Graph Measures (30 pts)

All measures below are computed on the graph built in Q3: the simple undirected graph read from `graph.txt` (one `networkx.Graph`, one node per distinct id, one edge per line), which has 26,728 nodes, 26,377 edges and 4104 connected components, the largest of which contains 13,903 nodes and 16,695 edges. All computations use NetworkX with its default parameters; every number was reproduced under NetworkX 3.3 and 3.6.1 and independently re-derived with SciPy (see the remarks).

1.1 Eigenvector centrality (10 pts)

`nx.eigenvector_centrality(G)` was called with default parameters (`max_iter= 100`, `tol= 10-6`). **It converged**; no `PowerIterationFailedConvergence` was raised, so the default call is the one reported. Values are the entries of the principal eigenvector of the adjacency matrix, normalised to unit L^2 norm.

Rank	Node	Degree	Centrality (default)	Centrality (exact)
1	16485	57	0.491844	0.492371
2	22777	59	0.419863	0.422581
3	14618	38	0.177656	0.177494
4	21928	17	0.139183	0.139695
5	2884	29	0.133813	0.133861
6	1197	18	0.133186	0.133738
7	9068	51	0.111138	0.102654
8	17902	32	0.109219	0.109103
9	1324	20	0.100115	0.100053
10	25747	19	0.094368	0.094399

Remark on the default tolerance. The fourth column is the answer with default parameters, as asked. For comparison the fifth column is the exact eigenvector from `eigenvector_centrality_numpy`. The two agree on the *set* of top-10 nodes, but they order ranks 7 and 8 differently: the exact computation puts node 17902 ahead of node 9068. The reason is that NetworkX's power iteration stops when the total L^1 change falls below

$$n \cdot \text{tol} = 26,728 \times 10^{-6} \approx 2.67 \times 10^{-2},$$

which is a loose *absolute* bound on a vector of unit norm; the returned vector has residual $\|Ax - \lambda x\|_\infty \approx 7.8 \times 10^{-3}$ and Rayleigh quotient 8.76202 against the true $\lambda_{\max} = 8.762217$. Node 9068 is the entry still furthest from its limit, so its rank has not settled after 100 iterations. Running the same power iteration with `max_iter= 105`, `tol= 10-10` reproduces the exact column to 9.3×10^{-7} .

Interpretation. Because the graph is disconnected, the principal eigenvector is supported on the component with the largest spectral radius. Here that is the giant component ($\lambda_{\max} = 8.7622$, versus at most 3.63 for every other component), so all ten top nodes lie in the giant component and roughly half the nodes (12,908, or 48.3%) receive centrality below 10^{-12} . Note also that eigenvector centrality is not simply degree: only 6 of these 10 nodes are also in the top 10 by degree. Node

21928 ranks 4th with degree just 17, because its neighbours are themselves central, while node 17546 has degree 40 yet does not make the list.

1.2 Katz centrality (10 pts)

`nx.katz_centrality(G)` was called with default parameters ($\alpha = 0.1$, $\beta = 1.0$, `max_iter`= 1000, `tol`= 10^{-6}), solving $x = \beta \mathbf{1} + \alpha Ax$ and normalising to unit L^2 norm.

Admissibility of the default α . Katz centrality is well defined only for $\alpha < 1/\lambda_{\max}$. For this graph $\lambda_{\max} = 8.762217$, so $1/\lambda_{\max} = 0.114126$. The default $\alpha = 0.1$ satisfies $0.1 < 0.114126$, so it is admissible (though not by a large margin). Consistently with this, the default call **converged** and every entry of the unnormalised solution is positive.

Rank	Node	Degree	Katz centrality
1	16485	57	0.211609
2	22777	59	0.175512
3	9068	51	0.109482
4	14618	38	0.099380
5	2884	29	0.074102
6	17902	32	0.070590
7	17546	40	0.068771
8	21928	17	0.066834
9	1197	18	0.061642
10	9740	27	0.060545

These values were confirmed against `nx.katz_centrality_numpy(G)` and against a direct sparse solve of $(I - \alpha A)x = \beta \mathbf{1}$; all three give the same ordering, agreeing to 1.3×10^{-5} and 1.9×10^{-16} respectively. Unlike eigenvector centrality, the $\beta \mathbf{1}$ term gives every node a positive baseline, so Katz does not collapse to zero off the giant component, and its ranking tracks degree more closely (node 9068, degree 51, rises from 7th to 3rd).

1.3 Average clustering coefficient and average Jaccard similarity (10 pts)

Average clustering coefficient. Using `nx.average_clustering(G)`, i.e. the mean over *all* 26,728 nodes of the local clustering coefficient $c_u = 2T_u/(d_u(d_u - 1))$, where T_u is the number of triangles through u , with the standard convention that $c_u = 0$ for nodes of degree < 2 :

$$\bar{C} = 0.100539.$$

This average is held down by the graph's sparsity: 13,942 of the 26,728 nodes (52.2%) have degree < 2 and so contribute 0 by convention, and the graph contains only 1926 triangles in total. Restricted to the 12,786 nodes of degree ≥ 2 the mean local clustering is 0.210168.

Average Jaccard similarity of all connected node pairs. For every edge $(u, v) \in E$ the Jaccard similarity of the two neighbourhoods is

$$J(u, v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|},$$

computed with `nx.jaccard_coefficient(G, G.edges())` and averaged over all 26,377 edges. Here $N(\cdot)$ denotes the *raw* neighbour set, so $v \in N(u)$ and $u \in N(v)$; consequently an isolated edge (both endpoints of degree 1) has $|N(u) \cap N(v)| = 0$ and scores $J = 0$ rather than being undefined. Under this convention

$$\bar{J} = 0.041304.$$

The value is small because 21,167 of the 26,377 edges (80.3%) join two nodes with no common neighbour and therefore score exactly 0; the largest value attained by any edge is $2/3$. (For contrast, excluding the two endpoints from each other's neighbour sets would give 0.084305; the number reported above is the NetworkX convention stated in the formula.)